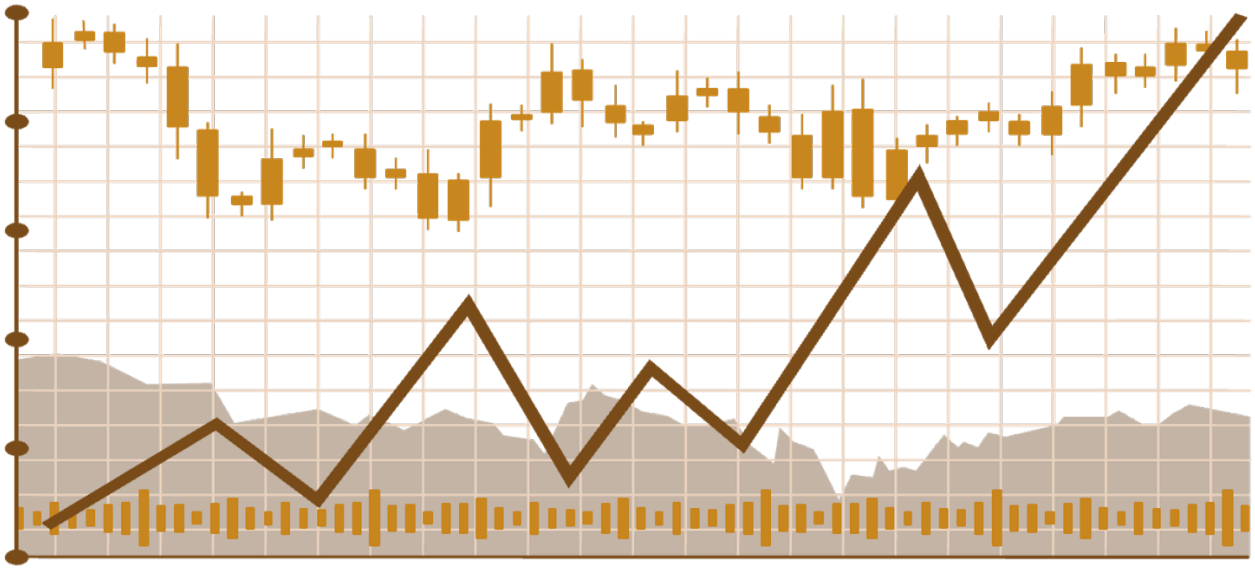
TSM-MachLeData
Beat Wolf
Bertil Chapuis
S1 - 2025-2026

Hes·so



Stock Price Prediction using Machine Learning

Advanced MLOps Topic: Data Drift

Justin Ferrara **Luca Da Silva** **Rafael Cardoso** **Thomas Maksimous**

10.12.2025

Table of contents

1 - Introduction 3
2 - Advanced Topic - Data Drift 4
  2.1 - Data Drift Detection 4
3 - Model Development and Deployment 5
  3.1 - Data engineering 5
  3.2 - Model development 5
  3.3 - Deployment 6
  3.4 - Model Evaluation Results 7
4 - Performance Improvements 8
  4.1 - Feature Engineering Enhancements 8
  4.2 - Model Architecture Optimization 8
  4.3 - Results 8
5 - Data Drift Analysis and Integration 8
  5.1 - Candidate Dataset 9
  5.2 - Data Drift Detection Strategy 9
  5.3 - Prepared Step Improvements 9
  5.4 - Pipeline Enhancements 9
  5.5 - Monitoring Approach 10
  5.6 - Results and Findings 10
6 - Conclusion 10
Bibliography 11
7 - Annexes 12
  7.1 - Selected Companies 12
  7.2 - Data Drift Reports 12

1 - Introduction

The objective of this project is to develop a simple predictive model capable of estimating future stock prices with within an MLOps framework. The model uses historical daily closing prices as input and predicts the closing prices for the following five days. The dataset consists of five years of historical data for several major technology companies listed on the National Association of Securities Dealers Automated Quotations (NASDAQ) index.

As part of the advanced MLOps component, the project focuses on analysing data drift and its impact on model performance. Each week, we perform a data drift analysis and retrain the model using newly collected data. The performance of the newly created model is then compared to older versions. This approach allows us to evaluate whether previously trained models degrade over time as market conditions evolve. The goal is to understand how data drift affects predictive accuracy and to highlight the value of continuous monitoring and retraining within an MLOps workflow.

The results of this project will provide insights into the challenges of stock price prediction and the importance of managing data drift in machine learning systems deployed in dynamic environments.

The contributions of each team member are as follows:
- Justin Ferrara: Data acquisition, preprocessing, initial model development.
- Luca Da Silva: Data drift research, pipeline integration, report writing.
- Rafael Cardoso: Research on various topics and report writing.
- Thomas Maksimous: Report writing and making presentations.

2 - Advanced Topic - Data Drift

The following section provides an in-depth explanation of model drift, data drift, feature drift, and label drift, along with methods for detecting data drift in machine learning systems [1], [2], [3], [4], [5].

In machine learning systems, **model drift** (also known as concept drift) refers to the degradation of a model's predictive performance over time due to changes in the underlying data distribution or relationships between input and output variables. This phenomenon occurs because the model was trained on past data that may no longer represent current real-world conditions. In financial forecasting, for instance, market trends, investor behavior, or economic indicators may evolve, causing previously learned patterns to become outdated. Detecting and managing model drift is therefore critical to maintaining accuracy and reliability in predictive systems.

One key source of model drift is **data drift**, which occurs when the statistical properties of input features (the X variables) change over time. For example, if stock trading volumes or volatility levels shift due to external economic events, the distribution of input data no longer aligns with what the model originally learned. Data drift can be measured through statistical tests such as the Kolmogorov-Smirnov, a non-parametric method for comparing distributions or by comparing feature distributions between training and production data. Regular monitoring allows to identify when the input data environment has shifted significantly, prompting the need for model retraining or adjustment. [6]

A related but distinct phenomenon is **feature drift**, which occurs when the relevance or predictive power of certain input features changes relative to others. For example, a feature like "daily trading volume" might become less predictive of price movement if market liquidity conditions change. Feature drift affects model interpretability and can mislead the model's weighting of inputs, even if overall data distributions appear stable. Detecting feature drift often involves monitoring metrics like feature importance or mutual information across time.

Finally, **label drift** (or target drift) refers to a change in the distribution of the target variable (Y). In stock prediction, this could occur if the overall market sentiment changes and the expected returns follow a different trend than before. Label drift can appear independently of input changes. For instance, when a company changes its dividend policy or experiences sudden external shocks. Continuous evaluation on recent labeled data helps detect this drift and guides the decision to retrain or recalibrate the model.

Together, these forms of drift—model, data, feature, and label, highlight the importance of dynamic monitoring and adaptation in MLOps. Properly managing them ensures that machine learning models remain accurate, fair and robust in evolving real-world environments.

2.1 - Data Drift Detection

Data drift detection aims to identify changes in the statistical properties of the input features over time, which may lead to a degradation in model performance if not properly monitored. Several families of methods can be used to detect such shifts. Some common approaches include:

- Statistical tests: they compare the distributions of features between the training data and more recent production data. Tests such as the Kolmogorov-Smirnov test or the Chi-squared test can be applied to continuous and categorical variables respectively, to identify statistically significant distributional changes [1], [2].
- Distance metrics: Measures such as Kullback-Leibler divergence or Jensen-Shannon divergence quantify the difference between probability distributions of features over time [1], [2].
- Visualization: Tools like histograms, box plots, or density plots provide an intuitive way to inspect how feature distributions evolve and to complement quantitative metrics with visual evidence [4].

- Machine learning-based methods: Machine learning-based methods can be used to model the normal behaviour of the data and flag deviations as potential drift; for example, unsupervised models such as autoencoders can learn typical patterns in the training data and highlight anomalous inputs that diverge from these patterns [4].

3 - Model Development and Deployment

This section presents the first version of the ML model, covering the data engineering pipeline, the machine learning model, its deployment within an MLOps workflow and the evaluation of its performance.

3.1 - Data engineering

The data engineering phase is a critical foundation for the stock price prediction task. It ensures that the data is correctly collected, cleaned, transformed and stored so that it can be reliably used by the machine learning model.

3.1.1 - Data acquisition

To acquire the necessary data for stock price prediction, we used the nasdaq.com website to download historical stock data for 20 stocks in the NASDAQ index. The data collected is in CSV format, and contains columns such as Date, Close, Open, High, Low, and Volume for the past 5 years. The companies listed in Section 7.1 were chosen based on their market capitalization and relevance in the technology sector. The data spans from `11/10/2020` to `11/07/2025` for each stock, providing a comprehensive dataset for training and evaluating the machine learning models.

3.1.2 - Data preprocessing

The preprocessing steps aim to clean the raw data and transform it into a format suitable for time series modelling. We performed the following operations:

- Checking for missing values and handling them appropriately (either by removal of incomplete rows or, when necessary, by simple imputation strategies).
- Converting the closing prices to float data variables, representing the variation in percent from the previous closing price.
- Normalizing the numerical features so that they lie on comparable scales, which is important for neural network training stability.
- Sorting the data chronologically by date and splitting it into training, validation and test sets in a time-respecting manner, to evaluate the model's performance on unseen future periods.

3.1.3 - Data storage

The raw and pre-processed datasets are stored in CSV format and tracked with DVC for data version control. This setup makes it possible to reproduce experiments, roll back to previous dataset versions and keep data, code and model artifacts aligned over time.

The data is stored in a dedicated S3 bucket for easy access and scalability (as shown in the tutorial [7]).

3.2 - Model development

To develop the first version of the stock price prediction model, we followed the MLOps guide [7] to set up an end-to-end MLOps pipeline. This includes sequence construction, model training, evaluation and preparation for deployment.

3.2.1 - Feature engineering

We use a minimal feature engineering approach. Daily variations of closing stock prices (as explained in Section 3.1.2) are transformed into supervised learning sequences using a sliding-window method. This simple feature set serves as a baseline model.

3.2.2 - Model training and validation

The model used in this first version is a Long Short-Term Memory (LSTM) neural network designed for sequence-to-sequence regression. LSTMs are a type of recurrent neural network that incorporate memory cells and gating mechanisms, enabling them to capture temporal dependencies over longer sequences and to mitigate the vanishing gradient problem. This makes them particularly suitable for time series forecasting tasks [8].

The model is trained, with the features mentioned before. The training process optimizes the model parameters to minimize the prediction error on the training set, using Mean Squared Error (MSE) as the loss function and the Adam optimizer.

A validation set is used to monitor the model's performance during training, tune hyperparameters (such as the number of LSTM layers, hidden size, dropout rate and learning rate) and reduce the risk of overfitting.

3.2.3 - Model Evaluation

The model's performance is evaluated using metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R-squared (R^2). These metrics provide insights into how well the model predicts stock prices compared to actual values.

Here is a brief explanation of the evaluation metrics used:
- The Mean Absolute Error (MAE) measures the average absolute difference between predicted and actual values, showing how far predictions are from reality on average.
- The Root Mean Squared Error (RMSE) calculates the square root of the average squared errors, giving more weight to larger mistakes and highlighting significant prediction deviations.
- The R-squared (R^2) indicates how well the model explains the variance in the actual data, with values closer to 1 representing better predictive accuracy. [9]

The best and worst predictions are visualized using plots to illustrate the model's performance and to better understand its strengths and weaknesses.

3.3 - Deployment

To deploy the model, we use the MLOps guidelines to set up a base environment for deployment [7].

The model is initially trained locally. On every pull request, the full training pipeline is reproduced inside a Kubernetes pod using GitHub Actions and a self-hosted runner. Once the model is trained and validated, it is packaged with BentoML, containerised with Docker and deployed on Google Cloud Platform (GCP), following the approach described in Chapter 3.7 of the MLOps guide [7].

3.3.1 - CI/CD Pipeline

The CI/CD pipeline automates the lifecycle of model training, evaluation and deployment using GitHub Actions, DVC, CML and BentoML. It runs training workloads on a self-hosted runner in a Kubernetes cluster.

The workflow is triggered automatically when:

- Code is pushed to the main branch
- A pull request is opened

- The workflow is manually started from the GitHub Actions interface.

It consists of two main jobs:

1. Train and Report (`train-and-report`)

   Triggered on pull requests, this job automates model retraining and experiment reporting:

   - Checks out the repository and sets up Python `3.12` .
   - Installs dependencies and authenticates with Google Cloud.
   - Reproduces the training pipeline using DVC (`dvc repro --pull`).
   - Pushes updated model artifacts and metrics to the DVC remote storage.
   - Commits changes to `dvc.lock` when necessary.
   - Generates a CML report that compares the new experiment results with those of the main branch, including:
     - Parameter and metric differences (`dvc params diff` , `dvc metrics diff`).
     - Visual comparisons of plots (e.g. training history, prediction previews).
   - Publishes the report as a comment in the pull request for review.

2. Publish and Deploy (`publish-and-deploy`)

   Triggered when changes are merged into the main branch, this job:

   - Checks out the repository and installs dependencies.
   - Authenticates with Google Cloud and reproduces the final training pipeline via DVC.
   - Imports and builds the BentoML model artifact.
   - Containerises the Bento service and pushes the Docker image to Google Container Registry.
   - Retrieves credentials for the Google Kubernetes Engine (GKE) cluster.
   - Updates the Kubernetes deployment manifest to reference the new image version (tagged with the Git SHA).
   - Applies the updated deployment and service configuration on the cluster to roll out the new model version.

3.3.2 - Infrastructure

The serving infrastructure is hosted on Google Cloud Platform. Google Kubernetes Engine (GKE) is used to orchestrate the containerised BentoML service. This setup ensures that the model can be deployed in a consistent way across environments and benefit from Kubernetes features such as rolling updates and health checks.

The model itself is packaged as a BentoML service and built into a Docker image. This guarantees that all runtime dependencies (Python environment, libraries, model files) are bundled together, improving reproducibility and portability.

3.3.3 - Scalability

Because the model is deployed on GKE, it can be scaled horizontally by increasing the number of replicas of the serving pod. Kubernetes allows us to define resource requests and limits, as well as auto-scaling policies based on CPU or custom metrics. This makes it possible to handle varying workloads while keeping the application responsive, even during peak usage periods.

3.4 - Model Evaluation Results

The initial model's performance metrics on the test set are as follows:
- Mean Absolute Error (MAE): 1.87
- Root Mean Squared Error (RMSE): 2.45
- R-squared (R^2): 0.00

As shown in pull request [10], these results indicate that the model has limited predictive accuracy, with a relatively high average error in stock price predictions. The R^2 score of 0.00 suggests that the model does not explain any of the variance in the actual stock prices, highlighting the challenges of forecasting in such a volatile domain.

4 - Performance Improvements

As seen in the previous section, the initial model[10] was not performing well, with high error rates and low predictive accuracy. To address these issues, we implemented several performance improvements to enhance the model's capabilities, described in detail in the pull request [11].

4.1 - Feature Engineering Enhancements

To improve the model's performance, we extended the input data to include the past 60 days of stock prices instead of just 30 days. This provided the model with more historical context, allowing it to capture longer-term trends and patterns in stock price movements.

We also added new features derived from the existing data, to provide additional insights for the model. These features are described in the following list:

- **Short-term trend indicators:** Computed 5-day and 10-day simple moving averages (SMA_5, SMA_10) to smooth short-horizon fluctuations and capture underlying price trends.
- **Recent-information-weighted trend measure:** Incorporated a 5-day exponential moving average (EMA_5) to assign greater weight to recent observations and enhance sensitivity to short-term market movements.
- **Volatility approximation:** Derived a High-Low price spread feature as a proxy for intraday volatility.
- **Directional price-movement ratio:** Constructed an Open-to-Close price ratio to quantify daily directional bias.
- **Volume-based activity metric:** Calculated the percentage change in trading volume to characterize shifts in market activity.

4.2 - Model Architecture Optimization

To avoid overfitting and improve generalization, we increased the dropout rate in the LSTM layers from `0.1` to `0.2`. This adjustment helped the model to better handle unseen data by preventing it from relying too heavily on specific neurons during training.

4.3 - Results

The performance improvements led to significant enhancements in the model's predictive accuracy and reliability. The key results are summarized in the pull request [11], which includes:

- A reduction in MAE from 1.87 to 0.66.
- A decrease in R^2 score from 0.00 to −1.19 (which is not an improvement but expected due to the nature of stock price prediction).
- Visualizations comparing the model's predictions before and after the improvements, demonstrating better alignment with actual stock prices.

5 - Data Drift Analysis and Integration

To strengthen the robustness of the stock-prediction workflow, data drift detection was integrated directly into the MLOps pipeline with the pull request [12]. The `Evidently` package was added as the primary toolkit for measuring drift on the prepared datasets [13]. The package `Evidently` was chosen due to its comprehensive suite of drift detection methods, based on multiple algorithms depending on the column type [2].

5.1 - Candidate Dataset

The current dataset is the one used to train the active model, while the candidate dataset is the newly ingested data that will be used to retrain a new model. To generate the candidate dataset, the following options were considered:

- Adding the latest week's data to the current dataset.
- Adding the latest week's data while removing the oldest week's data to maintain a consistent dataset size.
- Using only the latest week's data as the candidate dataset.

The first option was selected to maximize the amount of data available for training the new model, while still allowing for effective drift detection by comparing the distributions of features between the current and candidate datasets. The second option can be considered if we don't want to increase the dataset size over time or if we want to focus on more recent data and avoid old macroeconomic conditions that may no longer be relevant. The third option was not selected as it would limit the amount of data available for training the new model, potentially leading to poorer performance.

5.2 - Data Drift Detection Strategy

The first step in our data drift detection strategy is to compare the statistical distributions of features between the current and candidate datasets. Using Evidently built-in statistical tests (e.g., Kolmogorov-Smirnov test for continuous features, Chi-squared test for categorical features), we assess whether significant changes have occurred in the input data over time.

The second step to detect data drift involves a dual-model strategy within the pipeline. The idea is to train and evaluate two models in parallel: a base model that remains fixed over time (based on initial data), and a new model, trained with the latest data (`candidate` model) on a weekly basis.

By comparing the performance difference between these two models over time, we can infer whether drift is present. When the fine-tuned model consistently outperforms the fixed base model on new data, it signals that the distribution has shifted and that continuous retraining is necessary for optimal predictive accuracy.

5.3 - Prepared Step Improvements

To facilitate effective drift detection, the data preparation step was enhanced to generate two distinct datasets:

- Candidate data: the newly prepared dataset generated using the policy described in Section 5.1 (folder `data/raw_candidate/`).
- Current data: the previously accepted dataset used for training the active model (folder `data/raw/`).

These datasets are stored in `data/prepared/` and `data/prepared_candidate/` respectively, allowing for direct comparison of feature distributions over time.

5.4 - Pipeline Enhancements

A new data drift detection step was added to the workflow. This step performs the following actions:

- Loads the current and candidate datasets.
- Uses Evidently to compute drift metrics across all features.
- Generates a HTML and JSON report summarizing the drift analysis.
- Inserts a summary of the JSON report into the pipeline comment containing:
  - Whether significant drift was detected for any features.
  - The number of drifted columns (features).
  - The number of columns analysed (features).

- Train both the base (current) and candidate models, allowing for performance comparison.
- Insert model performance metrics into the pipeline comment for both models.

This approach ensures that every potential model update is accompanied by a thorough drift analysis, enabling informed decisions about model retraining and deployment.

An example of the pipeline comment is shown in pull request [12]. The Figure 1 shows the generated report from the `Evidently` package on the original data without drift.

5.5 - Monitoring Approach

The monitoring is performed exclusively through the pipeline rather than a dedicated external monitoring service. This setup enables:

- Regular evaluation of drift through Evidently
- Tracking performance of both base and fine-tuned models
- Automated generation of drift reports as part of routine CI/CD operations

Although this approach ensures consistent oversight during the weekly data refresh cycle, it does not offer real-time monitoring or the granular diagnostics provided by specialized observability platforms. Nonetheless, it remains a practical approach within the project scope and supports effective detection of data drift and early signs of model degradation.

5.6 - Results and Findings

The integration of data drift detection into the MLOps pipeline provided valuable insights into the stability of the input data over time. However, the analysis revealed that no significant data drift was detected between the current and candidate datasets. Both models, the fixed base model and the fine-tuned candidate model performed similarly over time, indicating that the underlying data distribution remained relatively stable during the evaluation period. This might be attributed to the nature of stock market data, which can exhibit periods of stability interspersed with volatility. The pull request [12] contains detailed results and visualizations from the drift analysis.

To have a better comprehension of the data drift and to show a sample of the results obtained, a fake data drift has been implemented in a branch [14]. This allows to see how the data drift detection works in practice and to validate the effectiveness of the implemented approach. The Figure 2 shows an example of data drift detected using the fake data drift implementation.

6 - Conclusion

This project demonstrated how a complete MLOps workflow can support the development of a stock price prediction model. Although stock forecasting remains challenging and the model's predictive performance is limited, the improvements introduced—especially in feature engineering and model tuning —led to more stable results.

Data drift detection was successfully integrated into the pipeline using Evidently. While no real drift was observed in the available data, the artificial drift experiment confirmed that the system can identify and report distribution changes when they occur. This validates the monitoring approach and ensures that future updates can be assessed reliably.

The project's main limitations are the restricted feature set and the simplicity of the model architecture. Future work could include richer financial indicators, macroeconomic variables, more advanced models, and real-time monitoring. Despite these limitations, the project provides a solid foundation for an automated, drift-aware forecasting pipeline.

Bibliography

[1] Evidently AI, "What is data drift in ML, and how to detect and handle it." Accessed: Dec. 08, 2025. [Online]. Available: https://www.evidentlyai.com/ml-in-production/data-drift

[2] Evidently AI, "Data Drift - How data drift detection works." Accessed: Dec. 08, 2025. [Online]. Available: https://docs.evidentlyai.com/metrics/explainer_drift

[3] SymuFolk, "Detecting Data Drift - Tools, Techniques, and Best Practices for 2025." Accessed: Dec. 08, 2025. [Online]. Available: https://medium.com/@Symufolk/detecting-data-drift-tools-techniques-and-best-practices-for-2025-d0d34b863c3e

[4] Jayita Gulati, "Detecting & Handling Data Drift in Production." Accessed: Dec. 08, 2025. [Online]. Available: https://machinelearningmastery.com/detecting-handling-data-drift-in-production/

[5] Karyna Naminas, "Data Drift - Detection and Monitoring Techniques." Accessed: Dec. 08, 2025. [Online]. Available: https://labelyourdata.com/articles/machine-learning/data-drift

[6] GeeksforGeeks, "Kolmogorov-Smirnov Test (KS Test)." Accessed: Nov. 07, 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/kolmogorov-smirnov-test-ks-test/

[7] Swiss AI Center, "A guide to MLOps." Accessed: Nov. 07, 2025. [Online]. Available: https://mlops.swiss-ai-center.ch/

[8] Rebeen Hamad, "What is LSTM? Introduction to Long Short-Term Memory." Accessed: Dec. 03, 2023. [Online]. Available: https://medium.com/@rebeen.jaff/what-is-lstm-introduction-to-long-short-term-memory-66bd3855b9ce

[9] Kavya Srirelangi, "Understanding Model Evaluation Metrics for Machine Learning." Accessed: June 29, 2024. [Online]. Available: https://medium.com/@kavyasrirelangi100/understanding-model-evaluation-metrics-for-machine-learning-160d385b72c5

[10] JustinFerrara14, "Stock-ML Repository - Change to financial data PR 6." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/6

[11] JustinFerrara14, "Stock-ML Repository - Performance Improvements PR 10." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/10

[12] JustinFerrara14, "Stock-ML Repository - Data Drift Integration PR 12." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/pull/12

[13] Evidently AI, "Data Drift - Overview of the Data Drift Preset.." Accessed: Dec. 08, 2025. [Online]. Available: https://docs.evidentlyai.com/metrics/preset_data_drift

[14] JustinFerrara14, "Stock-ML Repository - Create Artificial Data Drift Branch." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML/tree/10-create-artificial-datadrift

[15] JustinFerrara14, "Stock-ML Repository." Accessed: Dec. 08, 2025. [Online]. Available: https://github.com/JustinFerrara14/Stock-ML

7 - Annexes

7.1 - Selected Companies

The following 20 companies were selected from the NASDAQ index for stock price prediction:

- Apple Inc. (AAPL)
- AMD Inc. (AMD)
- Amazon.com, Inc. (AMZN)
- ASML Holding N.V. (ASML)
- Broadcom Inc. (AVGO)
- Cisco Systems, Inc. (CSCO)
- Alphabet Inc. (GOOGL)
- International Business Machines Corporation (IBM)
- Facebook, Inc. (META)
- Microsoft Corporation (MSFT)
- Micron Technology, Inc. (MU)
- Netflix, Inc. (NFLX)
- NVIDIA Corporation (NVDA)
- Oracle Corporation (ORCL)
- Palantir Technologies Inc. (PLTR)
- Qualcomm Incorporated (QCOM)
- Starbucks Corporation (SBUX)
- Tencent Holdings Ltd. (TCEHY)
- Tesla, Inc. (TSLA)
- Taiwan Semiconductor Manufacturing Company Ltd. (TSM)

7.2 - Data Drift Reports

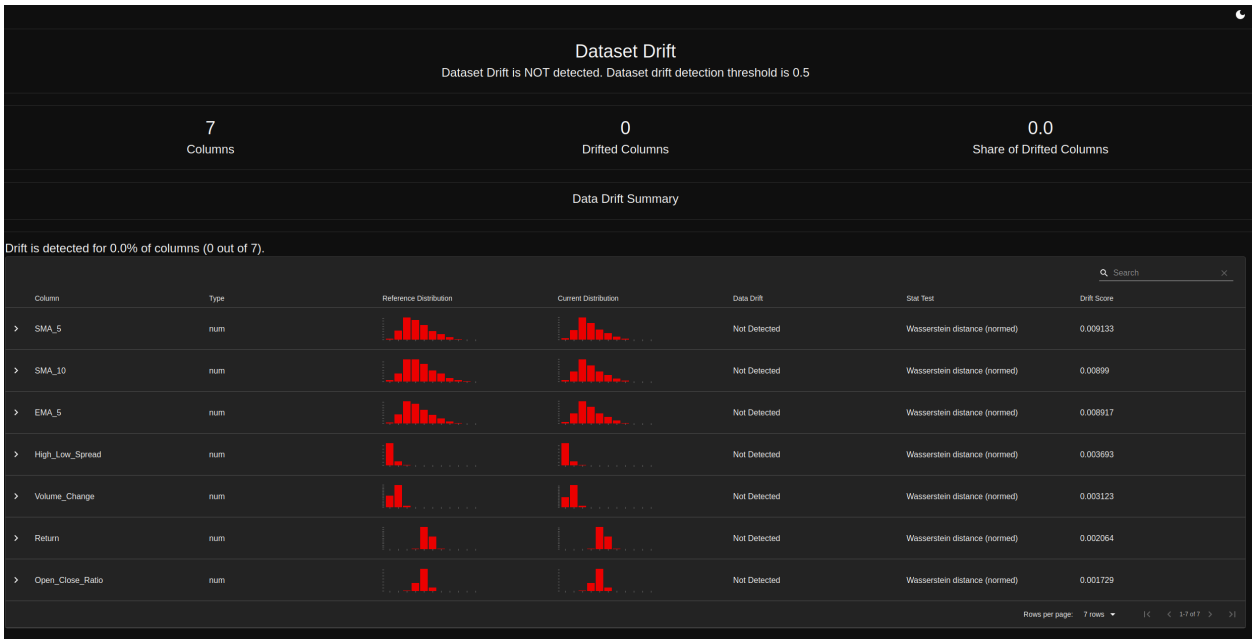


Figure 1 - Data Drift Report Example from Evidently

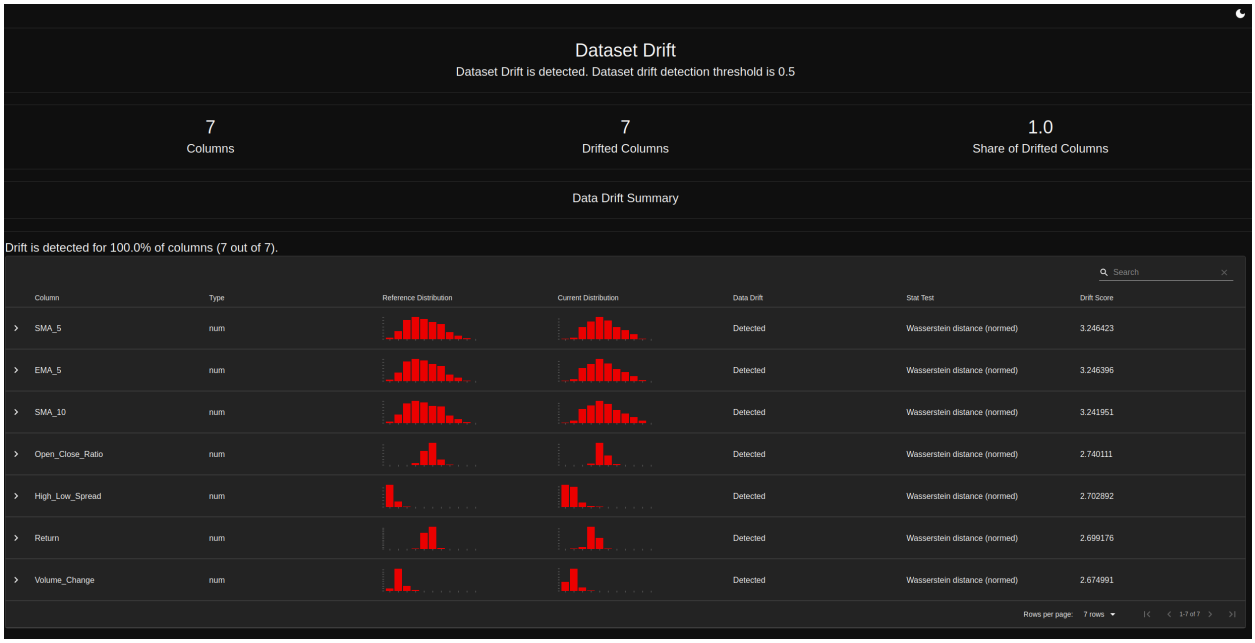


Figure 2 - Artificial Data Drift Report Example from Evidently